

Optimisation Numérique : Adam vs L-BFGS

KABLAM Edjabrou Ulrich Blanchard

Académie des Mathématiques Appliquées

15 août 2026

Table des matières

- 1 Introduction et mise en contexte
- 2 Concepts mathématiques et algorithmes
- 3 Programmation Python et applications

1. Introduction : Historique et Évolution

Contexte des méthodes d'optimisation

L'optimisation numérique est essentielle en sciences et en ingénierie pour minimiser des fonctions de coût $f(\theta)$ par rapport à des paramètres $\theta \in \mathbb{R}^n$ [1].

- **Années 1950-1970 (L'ère classique)** : Naissance de la descente de gradient, puis des méthodes quasi-Newton (Broyden 1965, DFP 1959, et BFGS en 1970 découvert indépendamment par Broyden, Fletcher, Goldfarb et Shanno) [2].
- **Années 2011-2015 (L'ère adaptative pour le Big Data)** : Apparition d'AdaGrad (2011), RMSProp (2012) et enfin **Adam** (Adaptive Moment Estimation) par Kingma & Ba en 2015 à ICLR [1].

Definition (Complexité Algorithmique)

La complexité algorithmique mesure la façon dont les ressources (temps d'exécution ou espace mémoire) évoluent en fonction de la taille n du problème (nombre de paramètres) en utilisant la notation Grand O (O).

Definition (Méthodes de Premier et Second Ordre)

- **Premier ordre** : Utilisent uniquement le gradient $\nabla f(\theta)$ (la pente) [1].
- **Second ordre** : Utilisent ou approximent la matrice Hessienne $\nabla^2 f(\theta)$ (la courbure) [2].

Équation de mise à jour

La méthode de descente de gradient cherche à minimiser une fonction objectif en suivant la direction de la plus forte pente (le gradient négatif) [3] via l'itération :

$$x_{k+1} = x_k - \alpha \nabla f(x_k) \quad (1)$$

où $\alpha > 0$ représente le pas d'apprentissage (*learning rate*) [3].

- **Convergence** : Linéaire et souvent lente, en particulier dans les vallées étroites (problèmes mal conditionnés) où l'algorithme zoome en zigzaguant [3].
- **Complexité** : Très efficace en mémoire avec une complexité spatiale en $O(n)$, ce qui la rend hautement scalable pour le Deep Learning comparativement aux méthodes du second ordre [3].

1. La Méthode de Newton

Équation de mise à jour

La méthode de Newton cherche les points stationnaires ($\nabla f(x) = 0$) via l'itération [2] :

$$x_{k+1} = x_k - (\nabla^2 f(x_k))^{-1} \nabla f(x_k) \quad (2)$$

- **Convergence** : Quadratique locale [2].
- **Complexité** : Stocker et inverser la Hessienne $n \times n$ coûte $O(n^2)$ en mémoire et $O(n^3)$ en calcul, ce qui est explosif pour le Deep Learning [3].

2. Les Méthodes Quasi-Newton : BFGS

Approximation de la Hessienne inverse

Au lieu de calculer la Hessienne, BFGS l'approxime par une matrice B_k mise à jour par une correction de rang 2 [2] :

$$H_{k+1} = H_k + \frac{y_k y_k^T}{y_k^T s_k} - \frac{H_k s_k s_k^T H_k}{s_k^T H_k s_k} \quad (3)$$

avec $s_k = x_{k+1} - x_k$ et $y_k = g_{k+1} - g_k$.

- **Limite mémoire** : Stockage de la matrice en $O(n^2)$, soit 8 To pour 1 million de paramètres [3].

3. L-BFGS (Limited-memory BFGS)

Algorithme 3 : Direction finding (L-BFGS)

L-BFGS contourne le problème en ne stockant que les $3 < m < 20$ derniers vecteurs (s_i, y_i) (où $m \ll n$) et utilise la récursion à deux boucles (*two-loop recursion*) [2] :

$$\text{Complexité Mémoire : } O(m \cdot n) \quad (4)$$

- Pour $n = 10^6$ et $m = 10$, on passe de 8 To à seulement 80 Mo de mémoire [3].

4. L'Algorithme Adam : Origine et Objectif

Definition (Signification d'ADAM)

ADAM signifie **AD**Aptive **M**oment estimation (Estimation de Moments Adaptatifs) [1].

Par qui et pourquoi a-t-il été créé ? (Kingma & Ba, 2015)

Créé par Diederik Kingma et Jimmy Ba en 2015 [1], Adam a été conçu pour **combiner les meilleurs avantages de deux mondes** :

- **AdaGrad** : pour son efficacité redoutable sur les gradients éparses (*sparse gradients*) [1].
- **RMSProp** : pour sa robustesse dans les contextes en ligne (*online*) et non-stationnaires [1].
- **Qu'est-ce que ça fait ?** Il calcule des taux d'apprentissage adaptatifs individuels pour chaque paramètre du modèle à partir des estimations des premier et second moments des gradients [1].

4. L'Algorithme Adam (Kingma & Ba, 2015)

Algorithme 1 : Mises à jour des moments

Adam combine momentum et taux adaptatifs [1] :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (5)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (6)$$

Correction du biais et mise à jour

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (7)$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (8)$$

où $\epsilon = 10^{-8}$ est une petite constante de stabilisation ajoutée au dénominateur [1]. **Pourquoi l'utiliser ?** Elle est indispensable pour **éviter une division par zéro** au cas où le moment second $\hat{v}_t$ deviendrait nul ou extrêmement proche de zéro, garantissant ainsi la stabilité numérique de l'algorithme [1].

Comparaison : L-BFGS vs Adam

Table – Tableau comparatif des optimiseurs L-BFGS et Adam.

Caractéristique	L-BFGS	Adam
Type	Second ordre [2]	Premier ordre [1]
Données	Batch / Déterministe [3]	Stochastique / Mini-batch [1]
Mémoire	$O(m \cdot n)$ [2]	$O(n)$ [1]
Usage	Problèmes lisses, modérés [2]	Deep Learning massif [1]

Note : *Batch / Déterministe* = calcul du gradient sur tout le dataset sans aucun hasard (très précis, mais consomme énormément de mémoire et de temps pour des millions de lignes).

Stochastique / Mini-batch = calcul par petits groupes aléatoires de données (introduit du bruit, mais accélère les mises à jour et évite de saturer le GPU ; le bruit est géré par les moyennes mobiles m_t et v_t d'Adam).

Comparaison Empirique : Adam vs L-BFGS

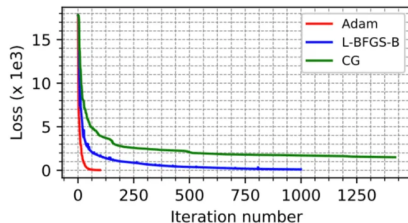
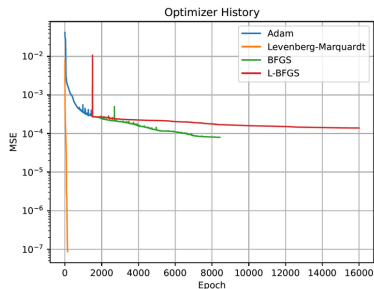


Figure – Évolution de la Loss et du MSE comparant Adam et L-BFGS sur des problèmes non-linéaires et réseaux profonds.

Sources des graphiques :

- Gauche : Sun et al., *A deep learning perspective of the forward and inverse problems in exploration geophysics* (2019) [4].
- Droite : Etude comparative sur les réseaux de neurones informés par la physique / deep learning (2020) [5].

Contexte des expériences

Ces graphiques comparent la vitesse de convergence des optimiseurs (notamment Adam et L-BFGS) sur des architectures de réseaux de neurones complexes et des problèmes d'optimisation non-linéaires [4, 5].

- **Observations (Graphique de gauche - MSE / Époque)** : Le modèle commence souvent par une phase d'entraînement avec Adam, puis bascule sur des méthodes de second ordre. On remarque la chute spectaculaire et ultra-rapide de l'erreur initiale pour Adam [5].
- **Observations (Graphique de droite - Loss / Itérations)** : Adam (courbe rouge) s'effondre vers une loss minimale en un nombre d'itérations extrêmement faible (≈ 100 itérations), surclassant nettement L-BFGS et le gradient conjugué (CG) [4].

- **Lequel est le meilleur et pourquoi ?**

- **Adam** est globalement supérieur en phase initiale et pour le Deep Learning massif : ses taux d'adaptativité par paramètre gèrent mieux le bruit stochastique des mini-batches.
- **L-BFGS** peut offrir une précision chirurgicale en fin d'optimisation sur des problèmes déterministes, mais s'avère beaucoup plus lent à converger au début en raison de l'estimation de sa courbure.

Annexe : Implémentation Python From Scratch (1/2)

```
import numpy as np
import time
import matplotlib.pyplot as plt

def fonction_cout(theta, X, y):
    m = len(y)
    return (1 / (2 * m)) * np.sum((X @ theta - y) ** 2)

def gradient(theta, X, y):
    m = len(y)
    return (X.T @ (X @ theta - y)) / m

def optimiseur_adam(X, y, theta_init, alpha=0.001, beta1=0.9, beta2=0.999, epsilon=1e-8,
    n_iter=100):
    theta = theta_init.copy()
    m = np.zeros_like(theta)
    v = np.zeros_like(theta)
    historique_cout = []

    for t in range(1, n_iter + 1):
        gt = gradient(theta, X, y)
        m = beta1 * m + (1 - beta1) * gt
        v = beta2 * v + (1 - beta2) * (gt ** 2)
        m_hat = m / (1 - (beta1 ** t))
        v_hat = v / (1 - (beta2 ** t))
        theta = theta - (alpha / (np.sqrt(v_hat) + epsilon)) * m_hat
        historique_cout.append(fonction_cout(theta, X, y))

    return theta, historique_cout
```


Annexe : Implémentation Python From Scratch (2/2)

```
def optimiseur_l_bfgs(X, y, theta_init, m_history=10, n_iter=100):
    theta = theta_init.copy()
    historique_cout = []
    s_list, y_list = [], []

    for k in range(n_iter):
        gk = gradient(theta, X, y)
        historique_cout.append(fonction_cout(theta, X, y))
        if k == 0:
            pk = -gk # Premier pas en descente de gradient pure
        else:
            # Algorithme 3 : Recursion a deux boucles (Two-loop recursion)
            q = gk.copy()
            alphas = []
            for s, y_vec in reversed(list(zip(s_list, y_list))):
                rho = 1.0 / (y_vec @ s)
                alpha = rho * (s @ q)
                alphas.append(alpha)
                q -= alpha * y_vec

            # Matrice initiale H0 approchée par un scalaire
            gamma = (s_list[-1] @ y_list[-1]) / (y_list[-1] @ y_list[-1])
            r = gamma * q

            for (s, y_vec), alpha in zip(zip(s_list, y_list), reversed(alphas)):
                rho = 1.0 / (y_vec @ s)
                beta = rho * (y_vec @ r)
                r += s * (alpha - beta)
            pk = -r

        # Pas fixe ou line search simplifiée
        alpha_step = 0.01
```

Annexe : Benchmark, Mesure de Temps et Tracé

```
# --- Generation de donnees de test ---
np.random.seed(42)
X_synth = np.c_[np.ones(500), np.random.rand(500, 5)]
vrai_theta = np.array([2.0, -1.0, 3.5, 0.5, -2.0, 1.0])
y_synth = X_synth @ vrai_theta + np.random.randn(500) * 0.1
theta_0 = np.zeros(6)

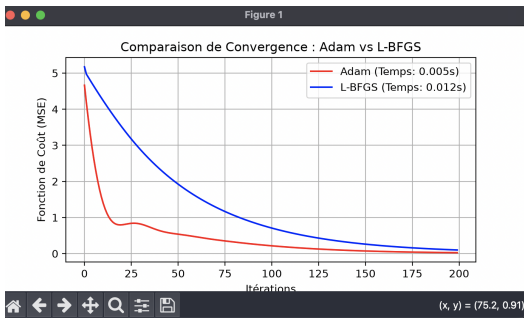
# --- Execution et Mesure du Temps pour Adam ---
t_start_adam = time.time()
th_adam, cout_adam = optimiseur_adam(X_synth, y_synth, theta_0, alpha=0.05, n_iter=200)
t_adam = time.time() - t_start_adam

# --- Execution et Mesure du Temps pour L-BFGS ---
t_start_lbfgs = time.time()
th_lbfgs, cout_lbfgs = optimiseur_l_lbfgs(X_synth, y_synth, theta_0, m_history=5, n_iter=200)
t_lbfgs = time.time() - t_start_lbfgs

print(f"Temps Adam : {t_adam:.4f}s | Temps L-BFGS : {t_lbfgs:.4f}s")

# --- Trace des graphiques comparatifs ---
plt.figure(figsize=(7, 3.5))
plt.plot(cout_adam, label=f"Adam (Temps: {t_adam:.3f}s)", color="red")
plt.plot(cout_lbfgs, label=f"L-BFGS (Temps: {t_lbfgs:.3f}s)", color="blue")
plt.xlabel("Iterations")
plt.ylabel("Fonction de Cout (MSE)")
plt.legend()
plt.title("Comparaison de Convergence : Adam vs L-BFGS")
plt.grid(True)
plt.show()
```

Résultat Pratique : Benchmark From Scratch (Adam vs L-BFGS)



1

1. Analyse des résultats *from scratch* : Sur un problème de régression synthétique de 200 itérations, Adam (courbe rouge) présente une chute de coût agressive et très rapide dès les premières itérations. L-BFGS (courbe bleue) décroît de manière plus linéaire et prudente, le temps d'accumuler assez d'historique de courbure (s_k, y_k). Côté performances, notre implémentation montre un avantage en vitesse brute pour Adam (0.005s contre 0.012s), illustrant le surcoût de la récursion à deux boucles en Python pur.

Références Bibliographiques



D. P. Kingma, J. Ba, *Adam : A Method for Stochastic Optimization*, ICLR 2015.



A. Gramfort, *Cours de Mathématiques Big Data : (Quasi-)Newton Methods*, Support de cours académique.



Synapsara, *Optimization Methods Series : BFGS & L-BFGS*, Capsule vidéo éducative.



J. Sun, Z. Niu, K. A. Inanen, D. Trad, *A deep learning perspective of the forward and inverse problems in exploration geophysics*, Conference Paper, April 2019.



ResearchGate Publication Figures, *Optimizing the optimizer for data driven deep neural networks and physics informed neural networks*, 2020.